

# GUIA DE ONBOARDING

## QA & Automação de Testes

Manual introdutório para pessoas que estão iniciando no setor de QA do Projeto Atual Mais, com foco em teste manual e automação com Playwright.

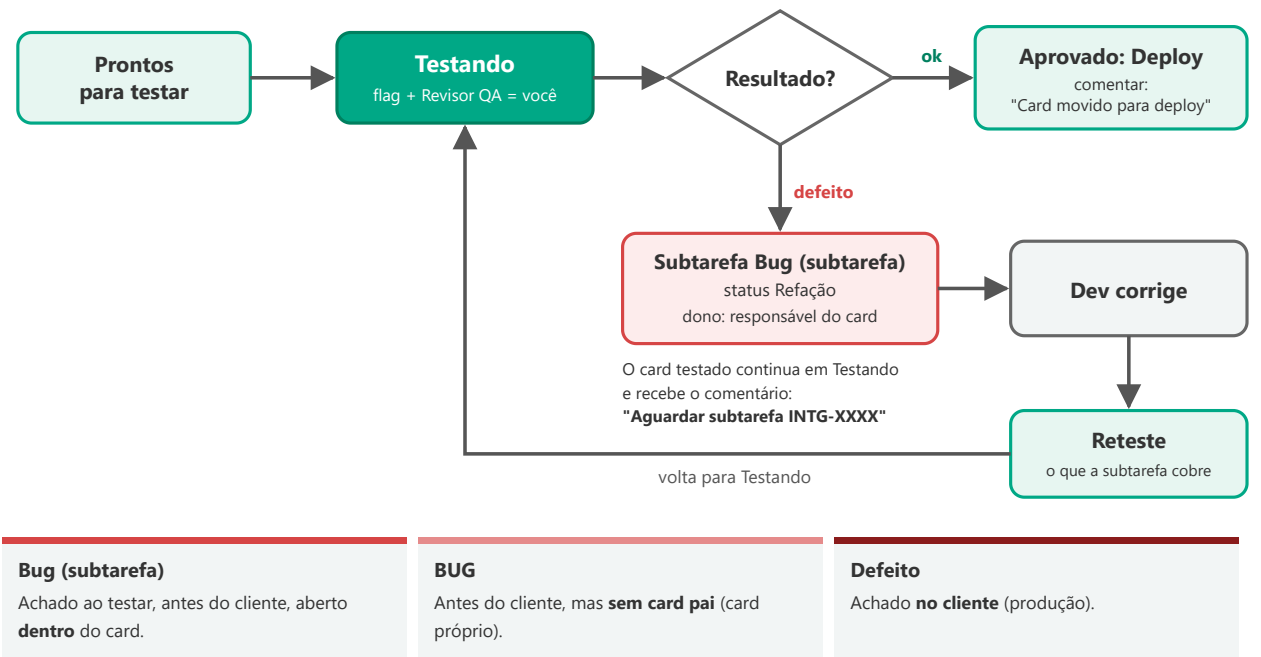
**ATUAL SOLUÇÕES**

Material interno · Setor de QA · Versão 1.8

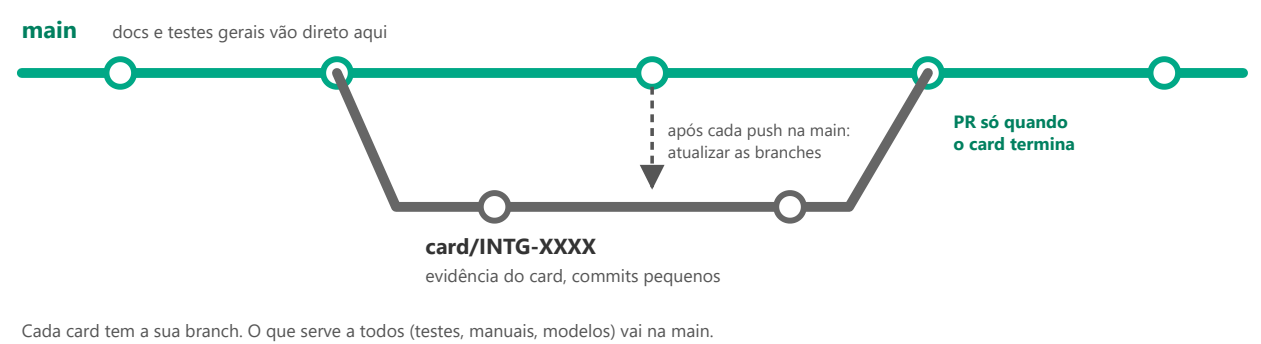
# Mapa visual do setor de QA (leia primeiro)

Este mapa mostra o trabalho em 5 desenhos. Os detalhes estão nas seções seguintes, e a **fonte sempre atualizada** são os documentos da pasta **docs/** do repositório.

## 1. O caminho de um card no Jira

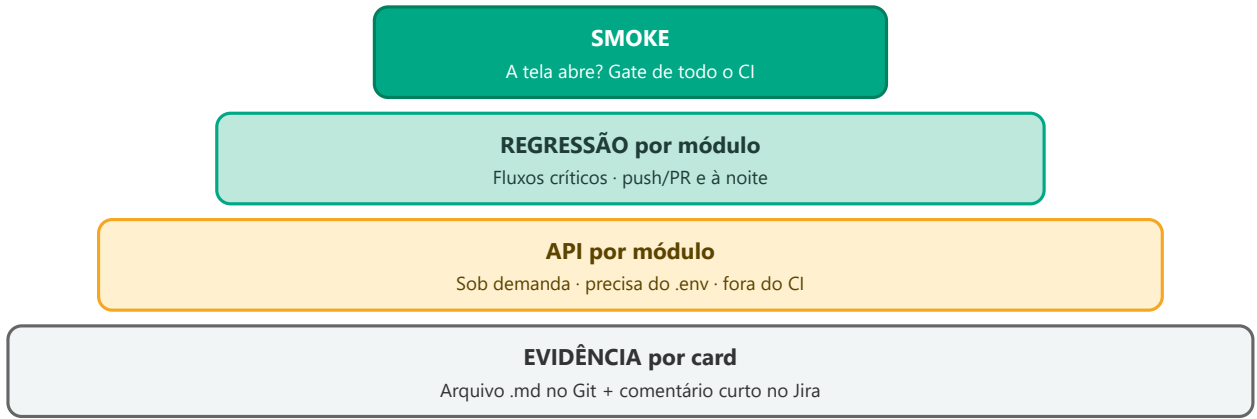


## 2. Como o Git é organizado



## Mapa visual: testes, pastas e CI

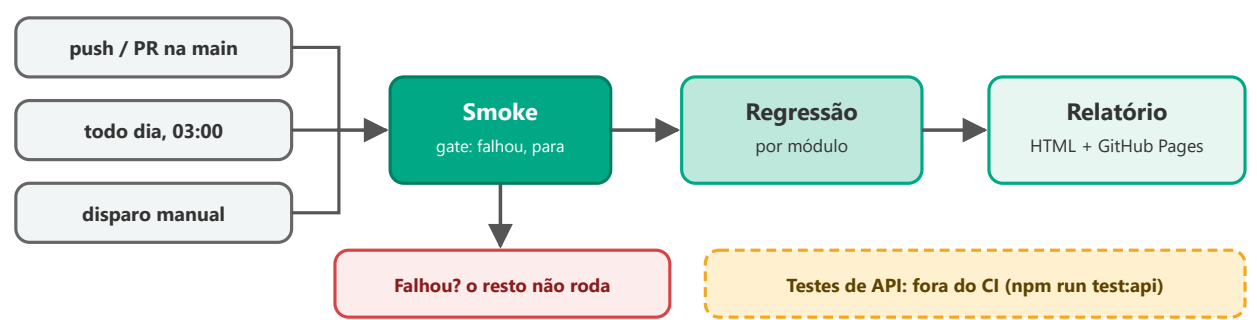
### 3. As camadas de teste



### 4. Onde cada coisa fica

|                            |                                                 |                                                |
|----------------------------|-------------------------------------------------|------------------------------------------------|
| tests/smoke/               | A tela abre?                                    | <code>@smoke</code> CI: sempre primeiro        |
| tests/regression/<módulo>/ | Regressão macro por módulo (vendas, financeiro) | <code>@regression</code> CI: push/PR e à noite |
| tests/api/<módulo>/        | Contrato do backend (hoje: financeiro)          | <code>@api</code> sob demanda, só em HMG       |
| tests/support/             | Login por API e ambiente, usados por todos      | helper compartilhado                           |
| tests/cards/               | Evidência de cada card e os modelos             | <code>npm run check:evidencias</code>          |
| docs/                      | Jira, Git, telas, falhas e testes de API        | fonte viva; este PDF resume                    |

### 5. O que o CI faz



**Regra de ouro:** o Jira guarda a **decisão e o resumo**; o detalhe fica **versionado no Git**. Fato provado é separado de hipótese, e nada de senha, token ou `.env` no repositório.

## 1. Objetivo do manual

Este manual apresenta a estrutura básica utilizada pelo setor de QA e Automação de Testes do Projeto Atual Mais: os ambientes de teste, as ferramentas do dia a dia, o fluxo de trabalho entre teste manual e automatizado, e a forma esperada de documentar e registrar as atividades.

Objetivo principal: dar autonomia rápida a quem está entrando no setor, reduzindo a dependência de explicações verbais e reunindo em um só lugar o necessário para começar a testar e automatizar com segurança.

**Versão em mapa mental:** este mesmo guia também está disponível em formato visual/interativo em [mm.tt/map/4095504042](https://mm.tt/map/4095504042).

## 2. Visão geral do setor de QA

O time de QA é responsável por validar as entregas do Atual Mais antes que cheguem à produção, cobrindo tanto testes manuais quanto testes automatizados end-to-end com Playwright.

O trabalho envolve: analisar cards e bugs reportados, desenhar cenários de teste, validar manualmente nos ambientes apropriados, automatizar os fluxos mais críticos e recorrentes, e documentar resultados de forma que qualquer pessoa do time consiga reproduzir o que foi encontrado.

O Atual Mais é um sistema ERP web (frontend React/Ant Design, via Umi) que cobre operações comerciais, fiscais, financeiras e administrativas — por isso, testes de fluxos fiscais e de venda exigem atenção redobrada.

## 3. Ferramentas utilizadas

As ferramentas abaixo fazem parte da rotina do setor de QA e possuem funções específicas no processo de teste e automação:

| Ferramenta           | Finalidade                                                                                    |
|----------------------|-----------------------------------------------------------------------------------------------|
| Jira                 | Gestão dos cards de teste, bugs encontrados e acompanhamento das entregas do time de QA.      |
| Clockify             | Registro do tempo dedicado a cada atividade de teste, manual ou automatizado.                 |
| Clarity              | Gerenciamento e controle de acessos utilizados nos ambientes de teste.                        |
| GitHub               | Versionamento e colaboração sobre o código do projeto de automação de testes.                 |
| GitHub Actions       | Pipeline de CI: roda a suíte de regressão em todo push/PR e a suíte completa todas as noites. |
| VS Code              | Editor utilizado para escrever, rodar e depurar os testes automatizados.                      |
| Playwright           | Framework de automação de testes end-to-end utilizado pelo time de QA.                        |
| JAM                  | Extensão do navegador para gravar o passo a passo e evidenciar bugs encontrados.              |
| DevTools (navegador) | Inspeção de elementos, seletores e chamadas de rede durante teste manual e automação.         |
| Claude               | Apoio na análise de HTML/seletores, escrita de testes e documentação das atividades.          |

Importante: nem toda automação substitui teste manual. Fluxos críticos (fiscal, pagamento, estoque) devem ser validados manualmente antes de serem automatizados.

## 4. Ambientes de teste

**HMG:** <https://backoffice.hmg.atualmais.com.br/> — ambiente inicial de validação. Todo card deve ser testado aqui antes de qualquer outra etapa.

**PREPROD:** <https://backoffice.preprod.atualmais.com.br/> — utilizado quando o card impacta operações críticas: vendas, entradas, tributos, orçamentos, OS e rotinas fiscais.

**Produção — Backoffice:** <https://backoffice.atualmais.com.br/>

**Link de teste em produção:** <https://company8.atualmais.com.br/>

Regra prática: HMG é o ambiente padrão de validação e de execução dos testes automatizados. A suíte Playwright (local ou no CI) nunca deve rodar contra produção.

## 5. Configurar o ambiente e preparar dados para testes fiscais

Antes de validar fluxos fiscais (NFC-e/NF-e), a empresa de teste no ambiente HMG precisa estar configurada. Isso só precisa ser feito uma vez por ambiente/empresa de teste:

- Solicitar o link do ambiente HMG, caso ainda não tenha.
- Em **Configurações > Editar > Dados Gerais**, preencher CNPJ e Inscrição Estadual da empresa de teste (ver tabela abaixo).
- Em **Configurações > Editar > Faturamento**, configurar NFC-e e NF-e conforme as tabelas a seguir.

### Configuração de NFC-e:

| Campo               | Valor                                     |
|---------------------|-------------------------------------------|
| Tipo                | 65 - Nota Fiscal do Consumidor Eletrônica |
| Forma de Emissão    | 1 - Normal                                |
| Ambiente            | 2 - Homologação                           |
| Tipo de impressão   | 0 - Geração de DANFE                      |
| ID Token            | 000001                                    |
| Token               | A8BEE0500FF3601DE0531E5FA00A9D26          |
| Série / Nota Fiscal | Usar numeração alta (ex.: 511)            |
| Versão do QRCode    | 2.0                                       |

### Configuração de NF-e:

| Campo               | Valor                          |
|---------------------|--------------------------------|
| Tipo                | 55 - Nota Fiscal Eletrônica    |
| Forma de Emissão    | 1 - Normal                     |
| Ambiente            | 2 - Homologação                |
| Tipo de impressão   | 1 - DANFE normal, Retrato      |
| Equipamento Fiscal  | 1 - satDinamico                |
| Série / Nota Fiscal | Usar numeração alta (ex.: 511) |

Dados de teste (empresa/certificado) usados na configuração acima e como massa de teste em cenários automatizados que envolvem NFC-e:

| Campo        | Valor                                      |
|--------------|--------------------------------------------|
| Token/ID     | 000001                                     |
| Token/CSC    | A8BEE0500FF3601DE0531E5FA00A9D26           |
| Razão Social | SG AUTOMAÇÃO COMERCIAL LTDA ME - ATUAL SGP |
| CNPJ         | 12.266.496/0001-29                         |
| IE           | 083.632.80-8                               |

- Sempre realizar os testes fiscais em homologação.
- Utilizar séries altas para testes, pois séries baixas podem estar sendo usadas por outras pessoas.
- O certificado digital deve ser solicitado a um integrante do time.
- Mesmo tendo Token/ID e Token/CSC, o certificado digital continua sendo necessário para determinados testes fiscais.

## 6. Fluxo de trabalho: teste manual e automação

- 1 Pegar só cards da coluna **Prontos para testar**; mudar a flag para **Testando** e se atribuir em **Revisor QA**. Conferir o número do card na URL antes de criar branch ou arquivo.
- 2 Mapear os cenários de teste em três tipos: **caminho feliz**, **negativo** e **borda** (ver seção 10).
- 3 Validar manualmente no ambiente HMG (e em PREPROD quando aplicável) e registrar o tempo no Clockify.
- 4 Registrar a evidência em **tests/cards/INTG-XXXX-evidencias.md** (modelo único, conferido por **npm run check:evidencias**) e comentar o veredito no card, em texto curto.
- 5 **Aprovado**: mover o card para deploy. **Defeito achado**: abrir uma subtarefa **Bug (subtarefa)**, atribuída ao responsável do card original, em **Refação**; o card testado continua em Testando, com o comentário "Aguardar subtarefa INTG-XXXX".
- 6 Avaliar se o cenário é candidato à automação (fluxo crítico, recorrente ou propenso a regressão). Teste de tela vai em **tests/regression/<módulo>/**; teste de API em **tests/api/<módulo>/**.
- 7 No Git, cada card tem a sua branch (**card/INTG-XXXX**); documentação e testes gerais vão direto na **main**, e o PR só abre quando o card termina.

Os detalhes ficam nos documentos vivos do repositório, que são a fonte: **docs/jira-fluxo-de-trabalho.md** e **docs/git-fluxo-de-trabalho.md**.

## 7. Estrutura do projeto de automação (Playwright)

O código de automação vive no repositório **github.com/romuloatual/atal\_mais**. A estrutura segue o padrão do Playwright Test: **camada primeiro, módulo depois**.

- **playwright.config.ts** — configuração global (baseURL padrão = HMG, navegadores, timeouts, relatórios).
- **tests/smoke/** — checagem rápida de que as telas principais abrem, tag **@smoke**. Roda primeiro em toda execução de CI, como gate antes do resto.
- **tests/regression/<módulo>/** — regressão macro por módulo (ex.: **vendas/**, **financeiro/**), e não por tela solta. Tag **@regression**.
- **tests/api/<módulo>/** — testes de API do módulo, tag **@api**. Rodam sob demanda: precisam do **.env** e criam dados no HMG.
- **tests/support/** — o que todos os módulos usam (ambiente, login por API).
- **tests/cards/** — evidências por card (**INTG-XXXX-evidencias.md**), modelos e o conferidor. O teste que sobra de um card vai para **regression/** depois de validado.
- **docs/** — Jira, Git, telas, falhas e testes de API; **package.json** — scripts npm.

Scripts npm principais:

|                                              |                                                      |
|----------------------------------------------|------------------------------------------------------|
| <code>npm test</code>                        | - roda a suíte, sem os testes de API (@api)          |
| <code>npm run test:headed</code>             | - roda os testes com o navegador visível             |
| <code>npm run test:ui</code>                 | - abre a interface interativa do Playwright          |
| <code>npm run test:report</code>             | - abre o último relatório de execução                |
| <code>npm run test:codegen</code>            | - grava ações no navegador e gera código de teste    |
| <code>npm run test:smoke</code>              | - roda só a suíte de fumaça (tag @smoke)             |
| <code>npm run test:regression</code>         | - roda só a suíte de regressão (tag @regression)     |
| <code>npm run test:api</code>                | - roda os testes de API (só em HMG, precisa do .env) |
| <code>npm run test:card -- @INTG-2645</code> | - roda só os testes de um card específico            |
| <code>npm run check:evidencias</code>        | - confere se as evidências seguem o modelo           |
| <code>npm run limpar:titulos</code>          | - prévia da limpeza de títulos de teste no HMG       |

Antes de escrever um teste de tela novo, rode **npm run test:codegen** no fluxo que você quer automatizar — ele já gera boa parte dos seletores e das ações.

## 8. Integração contínua (CI)

O workflow `.github/workflows/playwright.yml` automatiza a execução dos testes no GitHub Actions:

- **Sempre primeiro:** a suíte de fumaça (`@smoke`) roda como gate rápido — se as telas principais nem abrirem, o resto da suíte não chega a rodar.
- **Push/PR para main:** roda a suíte de regressão (`@regression`) depois do smoke — feedback rápido sobre se a alteração quebrou algo no mesmo módulo.
- **Todo dia às 03:00 (horário de Brasília):** roda a suíte completa, **sem os testes de API**, agendado (`schedule`).
- **Disparo manual (`workflow_dispatch`):** permite escolher entre rodar só a regressão ou a suíte completa.
- **Testes de API ficam fora do CI:** precisam de credenciais e a quitação deixa registros que o sistema não exclui. Rodam sob demanda, com `npm run test:api`.
- O relatório HTML de cada execução fica como artefato do workflow por 14 dias, e também publicado (sempre a versão mais recente) em [romuloatual.github.io/atual\\_mais](https://romuloatual.github.io/atual_mais) via GitHub Pages.

A CI nunca aponta para produção — o baseURL padrão do projeto é o ambiente HMG, o mesmo usado nas validações manuais.

## 9. Boas práticas para seletores e testes automatizados

- Prefira seletores estáveis: id (`#id`), role/texto visível (`getByRole`, `getByText`) e atributos data-testid quando existirem.
- Evite classes do CSS Modules com hash (ex.: `customerName__AxNXm`) — esse hash muda a cada build/deploy e quebra o teste sem que a funcionalidade tenha mudado.
- Ao mapear uma tela nova, use o DevTools (Copy > Copy outerHTML) para capturar o HTML já renderizado — o app é uma SPA (Umi/React), então o HTML estático da página não é suficiente.
- Para telas dentro de modais, popups ou iframes (ex.: pré-visualização de impressão), confirme se o elemento está dentro de um `#document` aninhado antes de tentar mapear os seletores.
- Documente qualquer inconsistência entre o esperado no card e o que a tela realmente mostra — isso pode indicar que o bug já foi corrigido parcialmente ou que o escopo mudou.

## 10. Como documentar um caso de teste e um problema

Regra de ouro: o **Jira guarda a decisão e o resumo**; o detalhe fica **versionado no Git**. Cada card tem um arquivo **tests/cards/INTG-XXXX-evidencias.md** no modelo único (**TEMPLATE-evidencias.md**), com os cenários em **BDD** (Dado / Quando / Então / Obtido), e o card recebe um comentário curto de veredito (**TEMPLATE-comentario-jira.md**). **Texto enxuto, evidência à vontade** (print, vídeo, payload).

Em cada card, cubra três tipos de cenário:

- **Caminho feliz (happy path)**: o que deve dar certo. Ex.: Telefone e Celular válidos aparecem no impresso.
- **Negativo**: dado inválido que deve ser barrado. Ex.: digitar letras em campo numérico; o campo não pode aceitar (depende da validação do dev).
- **Borda**: estourar o limite. Ex.: campo que aceita 10 caracteres, testar com 11.

Com muitos cards na fila, priorize 1 feliz + 1 negativo + 1 borda por campo ou regra.

```
Cenário N – [feliz / negativo / borda] [nome curto]
```

```
Dado    [estado inicial e dados]
```

```
Quando  [ação do usuário]
```

```
Então    [resultado esperado, conforme o card]
```

```
Obtido:   [o que aconteceu de fato] – passou / falhou
```

```
Evidência: [print, vídeo, payload ou PDF]
```

Ao encontrar um problema (Resultado Obtido diferente do Esperado):

- Abrir o card no **modelo de defeito do time** (**TEMPLATE-card-de-defeito.md**: Resumo, Objetivo, Regra de Negócio, Gherkin e Notas técnicas). Tipo **Bug (subtarefa)** se for do card em teste; **BUG** sem card pai; **Defeito** se foi achado no cliente.
- Informar o ambiente (HMG, PREPROD, Produção) e, se já existia, comparar com a produção e escrever "pré-existente".
- Adicionar evidências (prints, payload sem token, vídeo no JAM).
- **Separar fato de hipótese**: "provado por API ou payload" é uma coisa; "parece que" é outra. Nunca colar CPF, token ou senha.
- Em testes automatizados, linkar o relatório do Playwright ou o artefato publicado pela CI.

## 11. Checklist de entrada no setor de QA

- ☐ Acessar e conhecer o Jira do projeto.
- ☐ Configurar e utilizar o Clockify.
- ☐ Ter acesso ao Clarity.
- ☐ Ter acesso ao GitHub e clonar o repositório `github.com/romuloatual/atual_mais`.
- ☐ Ter o VS Code configurado, com a extensão Playwright Test for VSCode instalada.
- ☐ Instalar o Node.js e rodar `npm install` no projeto de automação.
- ☐ Rodar `npx playwright install` para instalar os navegadores de teste.
- ☐ Instalar/conhecer o JAM.
- ☐ Conhecer o uso do Claude como ferramenta de apoio.
- ☐ Conhecer os ambientes HMG, PREPROD e Produção.
- ☐ Conhecer a convenção de tags e de pastas (camada e módulo) e a pipeline de CI no GitHub Actions.
- ☐ Ler os documentos da pasta docs/ do repositório (fluxo do Jira, fluxo do Git, telas, falhas e testes de API).
- ☐ Para testes de API: criar o arquivo `.env` a partir do `.env.example` e preencher com o usuário de teste do HMG (o `.env` nunca vai para o Git).
- ☐ Conferir uma evidência de card com `npm run check:evidencias`.
- ☐ Solicitar acesso/certificado digital quando necessário para testes fiscais.
- ☐ Rodar a suíte de testes local pela primeira vez (`npm test`) e confirmar que passa.

## 12. Principais regras

- Todo teste, manual ou automatizado, deve estar vinculado a um card no Jira.
- O tempo das atividades deve ser registrado no Clockify.
- Testes automatizados (local ou CI) rodam por padrão contra HMG — nunca contra produção.
- Cada card tem a sua evidência em **tests/cards/INTG-XXXX-evidencias.md**; o teste automatizado que sobra só vai para **tests/regression/<módulo>/** depois de validado e considerado crítico/recorrente.
- O Jira guarda a decisão e o resumo; o detalhe fica versionado no Git. Fato provado é separado de hipótese.
- Nunca versionar senha, token ou **.env**. Testes de API só rodam em HMG e criam dados de teste que o sistema pode não excluir.
- Seletores baseados em classes com hash (CSS Modules) devem ser evitados na automação.
- Erros devem ser documentados de forma clara e reproduzível, com evidências anexadas.
- Dados de teste fiscal devem ser usados somente nos ambientes apropriados.
- Materiais e documentos do setor devem seguir o padrão visual da Atual Soluções.

Mentalidade esperada: entender o fluxo antes de testar, validar manualmente antes de automatizar, e deixar evidências suficientes para que qualquer pessoa do time consiga reproduzir o que foi encontrado.

### 13. Identidade Visual — Atual Soluções

A identidade visual dos materiais do setor deve seguir o padrão institucional da Atual Soluções, mantendo consistência nas apresentações, documentos e comunicações internas.

| Elemento            | Especificação                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| Verde institucional | PANTONE Green C · CMYK 100/0/65/0 · RGB 0/168/134 · HEX #00A886                                                       |
| Cinza institucional | PANTONE Cool Gray 10 C · CMYK 0/0/0/72 · RGB 102/102/102 · HEX #666666                                                |
| Tipografia          | Família sem serifa, com hierarquia clara entre títulos, subtítulos e texto.                                           |
| Estilo              | Visual limpo, corporativo e tecnológico, com boa hierarquia de informação e uso consistente das cores institucionais. |

Regra: as cores institucionais não devem ser alteradas arbitrariamente em materiais oficiais.